

空中具身智能挑战赛

技术报告

参赛队伍 永不坠机的曼波

团队成员

所属学校 浙江大学

参赛平台 微分智飞·非凸空间 C5 竞技级飞行机器人

参赛组别 标准组

提交日期 2026 年 5 月 25 日

目录

1	引言	1
1.1	时代背景与产业需求	1
1.2	核心技术与挑战	1
2	硬件平台	1
2.1	飞行平台概述	1
2.2	机载计算单元	1
2.3	飞控系统	2
2.4	传感器配置	2
2.5	通信与供电	2
2.6	安全机制	2
3	飞行机器人定位	3
3.1	概述	3
3.2	系统框架	3
3.3	IMU 前向传播与去畸变	4
3.4	点云特征匹配	4
3.5	IESKF 更新	5
3.6	iVox 增量体素地图	5
4	感知建图	6
4.1	概述	6
4.2	占用栅格地图	6
4.3	Raycast 射线追踪	7
4.4	障碍物膨胀	9
4.5	地图管理策略	11
5	轨迹规划	12
5.1	概述	12
5.2	系统框架	12
5.3	全局路径搜索	13
5.4	无 ESDF 轨迹优化	14
5.5	轨迹弹开重规划机制	16
5.6	安全保护机制	16
5.7	轨迹表示与优化	16
6	飞行控制	17
6.1	概述	17
6.2	控制系统架构	17
6.3	位置控制	17
6.4	姿态控制	18
6.5	偏航角控制策略	19
6.6	电机输出混合	19
6.7	实际飞行表现	20
7	系统级调参与实飞适配分析	20
7.1	调参背景与总体原则	20

7.2	控制与起降参数的适配修改	20
7.2.1	自动起飞高度与起降速度	20
7.2.2	电机预转时间与起飞后延时	21
7.2.3	状态切换重试间隔	21
7.3	经验证后选择沿用的关键参数	21
7.3.1	建图与膨胀参数的沿用	21
7.3.2	规划速度与安全距离参数的沿用	22
7.3.3	Faster-LIO 与外参参数的沿用	23
7.3.4	控制模型参数的沿用	23
7.4	参数之间的联动关系	23
7.4.1	起飞高度与虚拟墙	24
7.4.2	膨胀距离与可通行空间	24
7.4.3	速度上限与感知更新	24
7.5	本章小结	25
8	总结	25
8.1	系统方案总结	25
8.2	技术特点与工程价值	25
8.3	现阶段不足	25
8.4	后续优化方向	26

1 引言

1.1 时代背景与产业需求

低空经济作为战略性新兴产业的重要组成部分，正迎来政策与市场的双重驱动。我国“十五五”规划明确提出推进低空经济健康有序发展，为飞行机器人技术的产业化落地提供了核心指引。同时，飞行机器人作为低空经济的重要载体，在应急救援、城市安防、设施巡检、物流配送、农业植保等领域展现出巨大的应用潜力与广阔的市场空间。

然而，传统面向特定场景的专用化飞行机器人感知维度单一、作业模式固定，已难以适配城市复杂空间、野外林区等动态非结构化环境，导致其在处理多样化任务时存在效能瓶颈，无法达到产业应用深化的要求。当前，具身智能技术的发展成为推动飞行机器人从单一应用向通用化、高可靠作业转型的关键动能。面向真实场景的挑战，飞行机器人需要具备高度的环境适应性、运动灵活性与抗扰动能力，以完成各类高难度任务。破解飞行机器人在未知动态环境下的控制难题，成为行业发展的核心命题。

1.2 核心技术与挑战

面向复杂动态环境下飞行机器人通用化作业的产业需求，本技术围绕导航、规划、控制、系统集成四大核心环节，针对结构化障碍穿越、密集树林飞行、风扰稳定控制、移动障碍物躲避、狭窄区域穿越等典型工况，存在以下关键技术挑战：

- (1) **环境适应性与自主导航能力：**面对缺乏外部辅助导航信息的非结构化复杂场景，要求飞行机器人具备快速环境感知、理解与建模能力，实现自主定位与可靠导航，适应多样化地形与空间约束。
- (2) **动态避障与运动规划能力：**动态环境中的移动障碍物（如行人、车辆）、复杂地形约束对于飞行轨迹的安全性及高效性存在双重挑战，飞行机器人需具备毫秒级实时动态感知与最优轨迹规划能力，在保障飞行安全的前提下，计算最优飞行路径，实现复杂场景下的高效通行。
- (3) **抗扰动与精准控制能力：**风扰、气流变化等外部干扰会严重影响飞行机器人的飞行稳定性与作业精度，飞行机器人需具备强抗扰动能力与自适应调节能力，确保在复杂工况下保持稳定飞行姿态，实现精细的轨迹控制。
- (4) **系统集成能力：**通用化飞行机器人依赖多传感器与软硬件系统的高效协同，多源信息的有效融合、低延迟的数据处理与稳定的系统集成是保障机器人在复杂环境中可靠运行的关键支撑。

2 硬件平台

2.1 飞行平台概述

本系统采用微分智飞·非凸空间 C5 竞技级飞行机器人作为基础平台，采用高性能工程塑料与碳纤维轻量化设计，总重量约 1.2kg。平台预留丰富的传感器与计算单元接口，适合高性能飞行竞赛场景。

2.2 机载计算单元

核心计算单元采用英伟达 Jetson Orin Nano(67 TOPS INT8, 8GB 128-bit LPDDR5)，可实时运行 SLAM、路径规划等计算密集型算法，是本方案中性能功耗比最优的选择。



图 1 飞行平台实物图：C5 竞技级飞行机器人及其机载感知与计算组件

2.3 飞控系统

采用 NxtPX4v2 作为主飞控。飞控运行 PX4 开源固件，集成高精度 IMU、气压计、磁力计，支持 MAVLink/uORB 通信协议，具备故障自诊断与自动保护功能。PX4 生态成熟、社区活跃，便于快速开发和调试。

2.4 传感器配置

- (1) **激光雷达：**采用 Livox MID-360s，360° 水平视野，量程 40m，测距精度 $\leq 2\text{cm}$ 。相比传统机械式激光雷达，体积更小、功耗更低，适合机载应用。
- (2) **视觉传感器：**采用 RER-USB FHD01M 单目相机，配备水平 90° 镜头，分辨率 1080P@30fps，用于辅助定位与环境感知。
- (3) **惯性测量单元：**采用 BMI088/ICM-20602 双 IMU 配置，可有效抑制单一传感器的零点漂移和瞬时故障，提高姿态估计的鲁棒性。

2.5 通信与供电

通信系统采用双频 Wi-Fi (2.4GHz/5GHz) 实现机载与地面站的实时数据传输，同时保留 2.4GHz 无线电遥控链路作为备用控制方式。

供电系统采用 6S 4500mAh 锂电池，配合智能电源分配系统（具备过流、过压、欠压保护），典型载荷下续航约 14 分钟。

2.6 安全机制

系统设计一键停桨、碳纤维防护框、电池锁扣、地理围栏（虚拟边界）以及低电量/信号丢失自动返航等功能，为飞行机器人作业提供多重安全保护，确保飞行安全。

3 飞行机器人定位

3.1 概述

定位是飞行机器人实现自主导航的基础。在未知环境中，机器人需要实时估计自身的位姿（位置和姿态），并持续理解周围环境的几何结构。**Faster-LIO** 是一种典型的激光惯性里程计（LIO）方法，它将 IMU 的高频运动预测与激光雷达点云的几何约束紧密结合，在保证实时性的同时输出稳定、连续的位姿估计。

从处理流程上看，Faster-LIO 的核心可以概括为三步：先利用 IMU 完成高频状态预测，再对激光点云进行运动补偿与局部地图匹配，最后通过迭代误差状态卡尔曼滤波完成状态更新，并借助 iVox 增量体素维护局部地图。其优势在于计算链路清晰、模块耦合紧密，适合在机载算力受限的条件下运行。

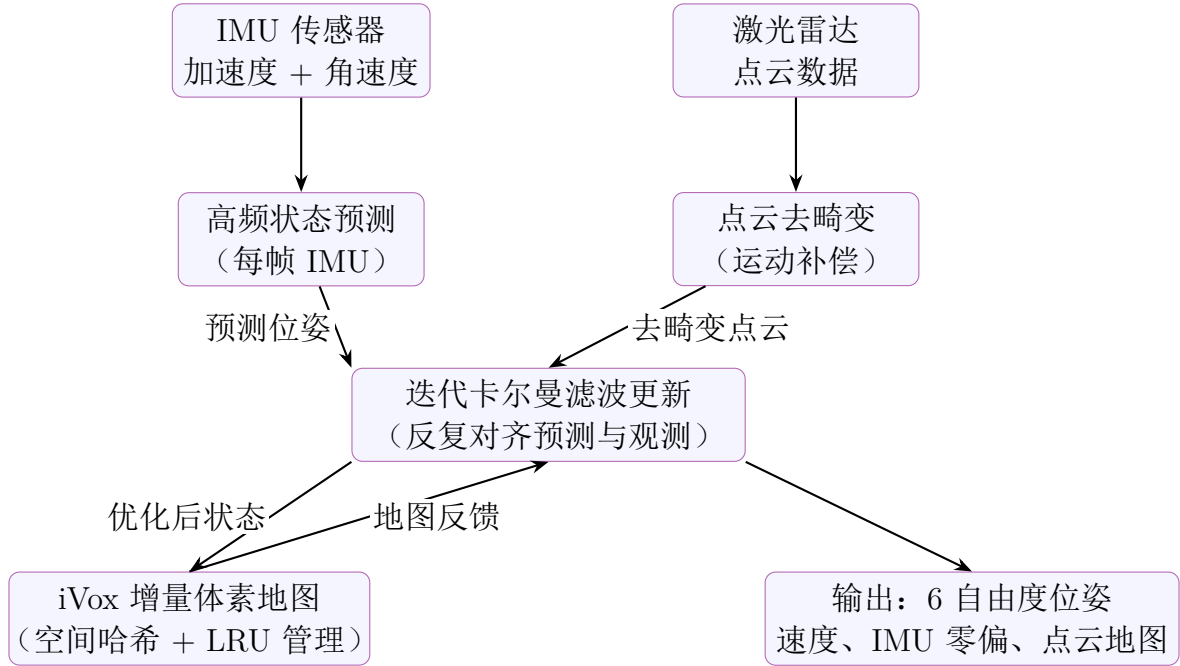


图 2 Faster-LIO 的典型数据流与核心处理链路

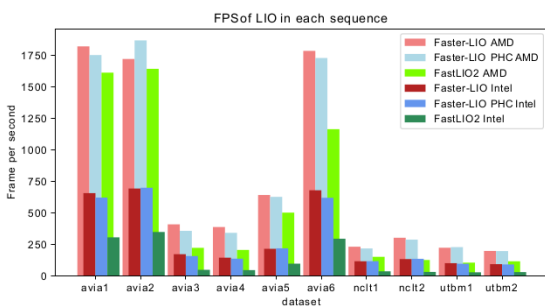


图 (a) Faster-LIO 与 Fast-LIO2 在不同雷达与计算平台上的运行帧率对比

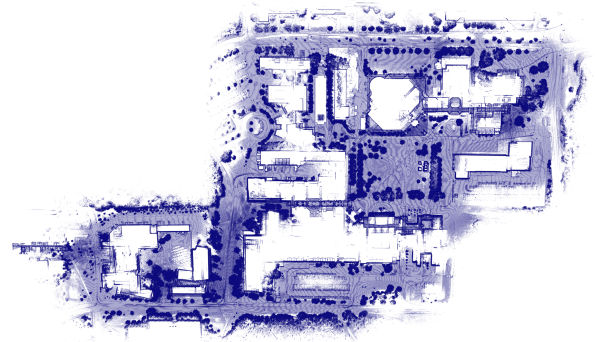


图 (b) Faster-LIO 在 NCLT 数据集上的位姿跟踪与点云建图效果

图 3 Faster-LIO 的工程验证示意：左为运行效率对比，右为真实数据集建图效果

图3 展示了 Faster-LIO 的性能和效果：左图说明其运行帧率相比于 Fast-LIO2 具有优势，右图表明算法能够在真实数据上稳定完成位姿跟踪与点云建图。

3.2 系统框架

Faster-LIO 采用 迭代误差状态卡尔曼滤波（IESKF）框架，系统状态定义在复合流形上：

$$\mathbf{x} = [\mathbf{p}^T, \mathbf{R}^T, \mathbf{R}_{LI}^T, \mathbf{t}_{LI}^T, \mathbf{v}^T, \mathbf{b}_g^T, \mathbf{b}_a^T, \mathbf{g}^T]^T \quad (1)$$

其中:

- $\mathbf{p} \in \mathbb{R}^3$: 位置
- $\mathbf{R} \in SO(3)$: 姿态旋转矩阵
- $\mathbf{R}_{LI} \in SO(3), \mathbf{t}_{LI} \in \mathbb{R}^3$: LiDAR-IMU 外参 (旋转与平移)
- $\mathbf{v} \in \mathbb{R}^3$: 速度
- $\mathbf{b}_g, \mathbf{b}_a \in \mathbb{R}^3$: 陀螺仪和加速度计的零偏
- $\mathbf{g} \in S^2$: 重力方向 (单位球面流形, 2 自由度)

IESKF 相比传统 EKF 的优势: 误差状态维度更低, 雅可比矩阵计算更简洁, 数值稳定性更好, 特别适合紧耦合的激光惯性融合问题。

3.3 IMU 前向传播与去畸变

IMU 提供高频 (200Hz) 的角速度和加速度测量。Faster-LIO 利用 IMU 数据完成两项任务: **状态前向传播**和**点云去畸变**。

对于两段激光雷达帧之间的 IMU 测量序列, 滤波器从上一帧的估计状态出发, 依次对每个 IMU 样本调用预测步, 使用中值积分将角速度和加速度测量融入状态:

$$\hat{\mathbf{x}}_{t+\delta t} = \mathbf{f}(\hat{\mathbf{x}}_t, \boldsymbol{\omega}_t, \mathbf{a}_t, \delta t) \quad (2)$$

其中 $\mathbf{f}(\cdot)$ 为连续时间运动学模型, $\boldsymbol{\omega}_t$ 和 $\mathbf{a}_t$ 为 t 时刻的 IMU 测量值。传播过程中记录每个 IMU 时刻的位姿 (位置、速度、姿态), 构成一条离散的位姿轨迹。

对于每个激光点, 根据其时间戳在 IMU 位姿轨迹中插值, 执行**反向补偿**: 将测量时刻的激光点变换到帧末参考时刻, 消除载体运动导致的点云畸变。具体地, 对于时间戳为 t_k 的激光点 $\mathbf{p}_{t_k}^L$, 补偿后的坐标在 LiDAR 坐标系下为:

$$\mathbf{p}_{\text{undistort}}^L = \mathbf{R}_{LI}^{-1} [\mathbf{R}_{t_e}^{-1} (\mathbf{R}_{t_k} (\mathbf{R}_{LI} \mathbf{p}_{t_k}^L + \mathbf{t}_{LI}) + \mathbf{t}_{t_k \rightarrow t_e}) - \mathbf{t}_{LI}] \quad (3)$$

其中 $\mathbf{R}_{t_k}, \mathbf{R}_{t_e}$ 分别为激光点时刻和帧末时刻的世界系姿态, $\mathbf{t}_{t_k \rightarrow t_e}$ 为位移。

这一流程将 IMU 的高频测量直接纳入滤波器的预测-校正框架, 无需显式构建帧间预积分量, 计算简洁且在实际中具有足够的鲁棒性。

3.4 点云特征匹配

激光雷达以 10Hz 频率提供环境的三维点云数据。为了提高配准效率, Faster-LIO 采用 **直接匹配策略**, 将当前帧点云通过当前位姿估计投影到局部地图坐标系, 并与 iVox 管理的局部地图进行最近邻搜索, 计算点到面的几何残差。

将当前帧点云通过位姿估计投影至世界坐标系, 在局部地图中搜索近邻点并拟合平面, 残差为点到平面的有符号距离:

$$r_i = \mathbf{n}_i^T \mathbf{p}_i^w + d_i \quad (4)$$

其中 $\mathbf{n}_i, d_i$ 为近邻点拟合的平面参数 ($\mathbf{n}_i^T \mathbf{p} + d_i = 0$), $\mathbf{p}_i^w$ 为激光点在世界坐标系下的坐标。

与传统特征提取方法的对比: LOAM 系列算法需提取边缘点和平面点, 对运动畸变敏感; Faster-LIO 的直接匹配策略省去了特征提取步骤, 在结构化与非结构化环境中均有稳定表现。

3.5 IESKF 更新

采用迭代误差状态卡尔曼滤波进行状态更新。滤波器先根据 IMU 前向传播预测状态与协方差，再用点云匹配残差修正状态；每轮迭代都在当前估计附近重新线性化，通常 2-3 轮即可收敛。其关键在于只在切线空间滤波误差状态 $\delta \mathbf{x}$ ，再通过 田 将修正量合入名义状态，从而避免直接加法破坏 $SO(3)$ 、 S^2 等流形约束。

第 κ 次迭代中，观测模型线性化为：

$$\mathbf{h}(\hat{\mathbf{x}}_k^\kappa \text{ 田 } \delta \mathbf{x}) \approx \mathbf{h}(\hat{\mathbf{x}}_k^\kappa) + \mathbf{H}_\kappa \delta \mathbf{x} \quad (5)$$

随后计算增益与修正量：

$$\mathbf{K}_\kappa = \mathbf{P}_k^- \mathbf{H}_\kappa^T (\mathbf{H}_\kappa \mathbf{P}_k^- \mathbf{H}_\kappa^T + \mathbf{R}_\kappa)^{-1} \quad (6)$$

其中 $\mathbf{H}_\kappa$ 为观测雅可比矩阵， $\mathbf{R}_\kappa$ 为观测噪声协方差。修正量 $\delta \hat{\mathbf{x}}_\kappa = \mathbf{K}_\kappa (\mathbf{z}_\kappa - \mathbf{h}(\hat{\mathbf{x}}_k^\kappa))$ 通过 $\hat{\mathbf{x}}_k^{\kappa+1} = \hat{\mathbf{x}}_k^\kappa \text{ 田 } \delta \hat{\mathbf{x}}_\kappa$ 合入状态；收敛后再更新协方差并重置误差状态。

Faster-LIO 使用 iVox 存储地图点云，并通过点到面 ICP 构造激光雷达观测。实际代码中的核心处理可概括为：并行投影当前帧点云，在收敛后刷新近邻和平面，对通过几何门控的点写入残差。

Listing 1: Faster-LIO 激光雷达观测模型 (laser_mapping.cc)

```

1 void LaserMapping::ObsModel(state_ikfom &s, esekfom::dyn_share_datastruct<double> &
   ekfom_data) {
2     std::vector<size_t> index(scan_down_body_>size());
3     std::iota(index.begin(), index.end(), 0);
4     auto R_wl = (s.rot * s.offset_R_L_I).cast<float>();
5     auto t_wl = (s.rot * s.offset_T_L_I + s.pos).cast<float>();
6     std::for_each(std::execution::par_unseq, index.begin(), index.end(), [&](const size_t
   &i) {
7         common::V3F p_body = scan_down_body_>points[i].getVector3fMap();
8         scan_down_world_>points[i].getVector3fMap() = R_wl * p_body + t_wl;
9         if (ekfom_data.converge) {
10             ivox_>GetClosestPoint(scan_down_world_>points[i], nearest_points_[i],
11                                   options::NUM_MATCH_POINTS);
12             point_selected_surf_[i] = common::esti_plane(
13                 plane_coef_[i], nearest_points_[i], options::ESTI_PLANE_THRESHOLD);
14         }
15         auto temp = scan_down_world_>points[i].getVector4fMap();
16         temp[3] = 1.0;
17         float pd2 = plane_coef_[i].dot(temp);
18         if (point_selected_surf_[i] && p_body.norm() > 81 * pd2 * pd2)
19             residuals_[i] = pd2;
20     });
21 }

```

这段实现保留了三个关键点：点级计算并行化；只在 `ekfom_data.converge` 后刷新近邻和平面；用 `p_body.norm() > 81 * pd2 * pd2` 过滤不稳定对应，避免坏残差进入滤波更新。

3.6 iVox 增量体素地图

为了高效管理大规模点云地图，Faster-LIO 采用 iVox (Incremental Voxel, 增量体素) 数据结构。与 Fast-LIO2 使用的 ikd-Tree 不同，iVox 用空间哈希和体素网格替代了树结构，这是 Faster-LIO 性能提升的关键创新点。

iVox 将三维空间按固定分辨率划分为体素网格，以网格整数坐标作为哈希键；近邻查询通过 CENTER、NEARBY6、NEARBY18、NEARBY26 等邻域模式收集候选点后排序。地图通过 LRU 链表管理体素访问顺序，容量超限时淘汰最久未访问体素；点云插入支持批量并行，体素创建与销毁为 $O(1)$ ，因此比树结构更适合机载实时维护。

4 感知建图

4.1 概述

感知建图是飞行机器人理解环境、实现自主避障的基础能力。本章介绍基于**占用栅格地图（Occupancy Grid Map）**的三维环境建模方法，以及采用 **Raycast 射线追踪** 技术实现的实时栅格更新流程。该模块将激光雷达点云和定位结果转化为可查询的局部占据地图，为后续轨迹规划提供碰撞检测、障碍物膨胀和动态环境维护能力。

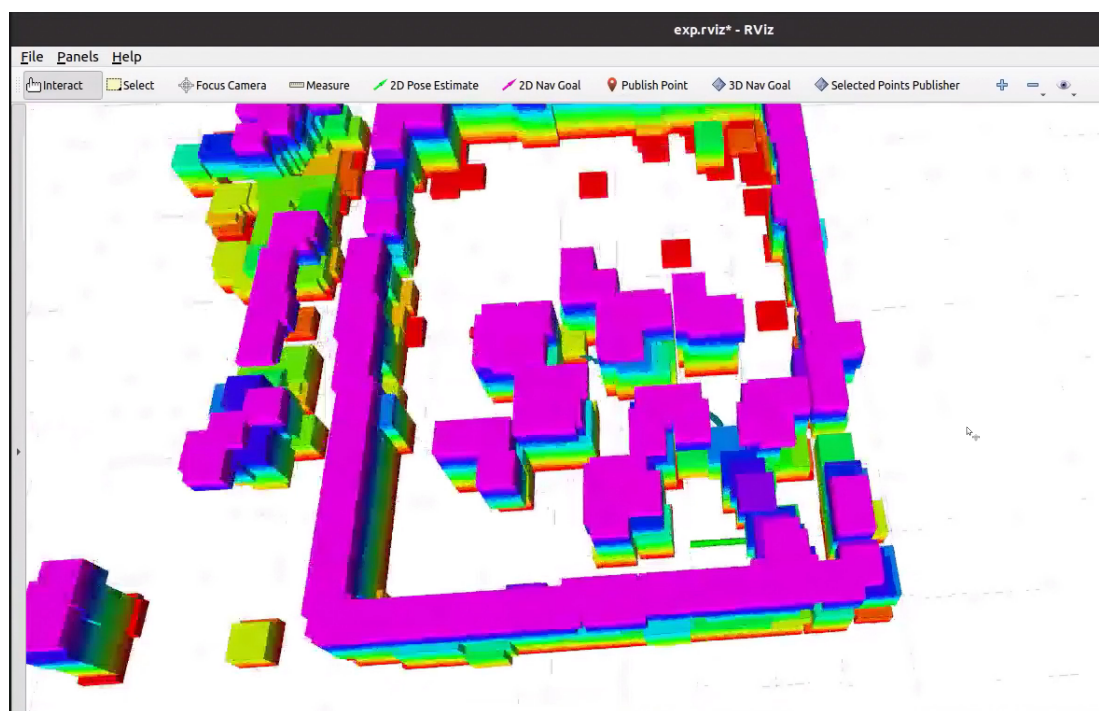


图 4 建图效果图：RViz 中显示的占用栅格与障碍物空间分布

4.2 占用栅格地图

占用栅格地图将环境离散化为规则的三维体素（Voxel），每个体素保存该空间位置被障碍物占用的概率。与直接维护原始点云相比，栅格地图更适合执行碰撞检测、障碍物膨胀和规划代价查询，因此常作为实时避障系统中的中间环境表示。

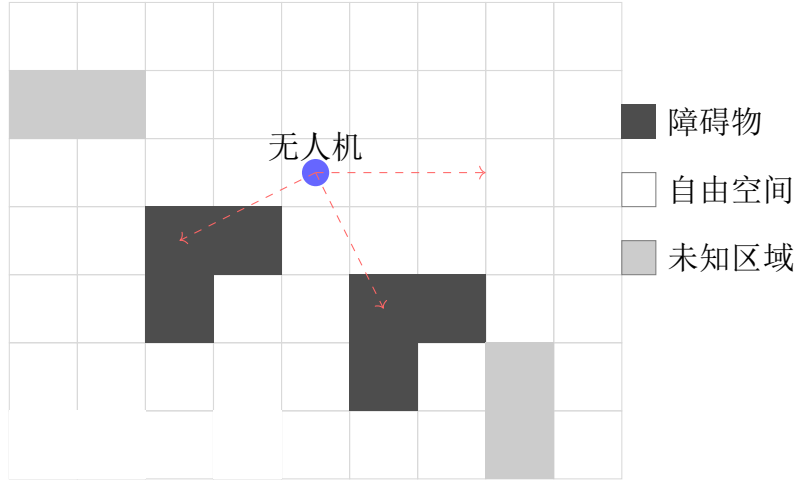


图 5 占用栅格地图示意：自由空间（白色）、障碍物（黑色）与未知区域（灰色），无人机通过激光射线更新地图

系统采用 贝叶斯更新框架维护体素状态。设 m_i 表示第 i 个体素的占用状态， $\mathbf{z}_{1:t}$ 表示截至 t 时刻的观测序列，则后验概率为：

$$p(m_i | \mathbf{z}_{1:t}) = \frac{p(\mathbf{z}_t | m_i) p(m_i | \mathbf{z}_{1:t-1})}{p(\mathbf{z}_t | \mathbf{z}_{1:t-1})} \quad (7)$$

为避免概率连乘带来的数值饱和问题，实际实现中采用 对数几率（log-odds）表示：

$$l_t(m_i) = \log \frac{p(m_i | \mathbf{z}_{1:t})}{1 - p(m_i | \mathbf{z}_{1:t})} \quad (8)$$

结合逆传感器模型 $p(m_i | \mathbf{z}_t)$ ，递归更新可写为：

$$l_t(m_i) = l_{t-1}(m_i) + \log \frac{p(m_i | \mathbf{z}_t)}{1 - p(m_i | \mathbf{z}_t)} - l_0(m_i) \quad (9)$$

其中 $l_0(m_i)$ 为先验对数几率；当先验占用概率取 0.5 时， $l_0(m_i) = 0$ 。为避免单个体素被少量误观测永久锁定，更新后还需对 $l_t(m_i)$ 进行上下界截断：

$$l_t(m_i) \leftarrow \min(l_{\max}, \max(l_{\min}, l_t(m_i))) \quad (10)$$

对数几率表示的优势：

- 通过加减法完成递归更新，适合高频（10Hz）实时建图
- 正负号含义直观： $l > 0$ 表示倾向占用， $l < 0$ 表示倾向空闲， $l = 0$ 表示未知
- 结合上下界截断后，可抑制数值发散并保留后续观测修正的空间

4.3 Raycast 射线追踪

Raycast 的核心思想来源于经典 SLAM 中的“命中-穿过”模型：从激光雷达原点向每个点云测量点发射一条射线，射线经过的体素被更新为空闲，射线终点对应的体素被更新为占用。相比全图遍历，Raycast 只访问观测相关的体素，因而更适合机载平台上的局部地图实时维护。

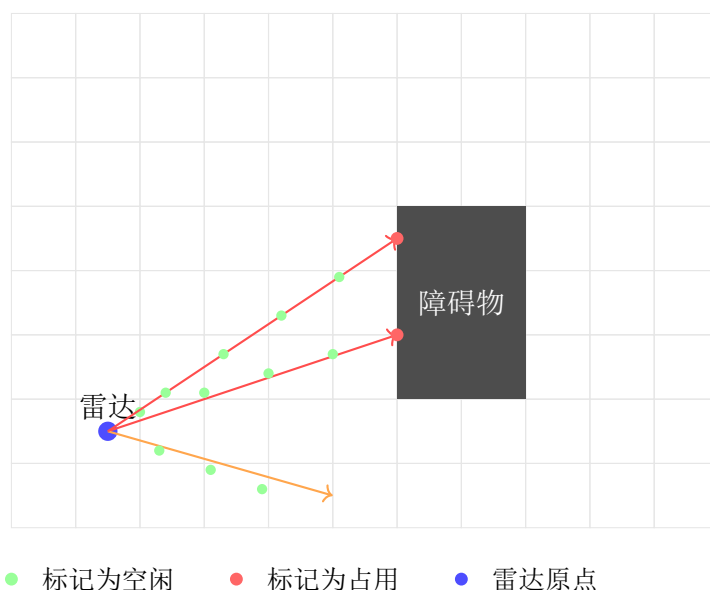


图 6 Raycast 射线追踪原理：雷达发射射线，路径上的体素标记为空闲，终点标记为占用

具体流程如下：

- (1) **坐标变换**：根据定位模块输出的位姿，将当前帧点云从雷达坐标系变换到地图坐标系。
- (2) **发射射线**：从雷达原点向每个有效点云返回点发射射线，并按地图分辨率遍历射线路径上的体素。
- (3) **更新空闲体素**：射线路径上、终点之前的体素被视为可通行空间，对应对数几率减小。
- (4) **更新占用体素**：射线终点所在体素被视为障碍物表面，对应对数几率增大。
- (5) **缓存合并**：同一帧中重复访问的体素先在缓存中统计命中/穿越次数，最后统一写回占用缓冲区。

与全图遍历方法相比的优势：

- **计算集中在观测区域**：复杂度与点数及每条射线穿过的体素数相关，可表示为 $O(\sum_i K_i)$ ，其中 K_i 为第 i 条射线穿越的体素数量，避免对整张地图做无效扫描
- **实时性强**：建图频率可达 10Hz，可与激光雷达数据帧率保持同步
- **利于局部地图维护**：射线遍历天然局限在传感器视野和有效测距范围内，适合滚动窗口地图

工程实现中，我们直接使用 Diff-Planner 的 GridMap。雷达点云进入 `cloudCallback()` 后，模块先检查里程计、过滤无效点，再依次移动环形窗口、执行 Raycast、更新膨胀地图。这个调用顺序保证算法描述与实际运行链路一致。

源代码路径：src/diff_planner/plan_env/src/grid_map.cpp

Listing 2: 建图入口与 Raycast 更新摘录 (grid_map.cpp)

```

1 void GridMap::cloudCallback(const sensor_msgs::PointCloud2ConstPtr &msg)
2 {
3     pcl::PointCloud<pcl::PointXYZ> latest_cloud;
4     pcl::fromROSMsg(*msg, latest_cloud);
5     if (!md_.has_odom_ || latest_cloud.empty()) return;
6     // filter finite points into md_.proj_points_
7     moveRingBuffer();
8     raycastFromCloud();
9     clearAndInflateLocalMap();
10    md_.occ_need_update_ = true;

```

```

11 }
12
13 void GridMap::raycastFromCloud()
14 {
15     if (md_.proj_points_cnt_ == 0) return;
16     md_.cache_voxel_cnt_ = 0;
17     md_.raycast_num_ += 1;
18     RayCaster raycaster;
19     Eigen::Vector3d ray_pt;
20     for (int i = 0; i < md_.proj_points_cnt_; ++i)
21     {
22         Eigen::Vector3d pt_w = md_.proj_points_[i];
23         int vox_idx;
24         if (!isInBuf(pt_w)) {
25             pt_w = closetPointInMap(pt_w, md_.camera_pos_);
26             vox_idx = setCacheOccupancy(pt_w, 0);
27         } else {
28             vox_idx = setCacheOccupancy(pt_w, 1);
29         }
30         if (vox_idx != INVALID_IDX) {
31             if (md_.flag_rayend_[vox_idx] == md_.raycast_num_) continue;
32             md_.flag_rayend_[vox_idx] = md_.raycast_num_;
33         }
34         raycaster.setInput(pt_w / mp_.resolution_, md_.camera_pos_ / mp_.resolution_);
35         while (raycaster.step(ray_pt))
36         {
37             Eigen::Vector3d tmp = (ray_pt + Eigen::Vector3d(0.5, 0.5, 0.5)) * mp_.resolution_;
38             int idx = setCacheOccupancy(tmp, 0);
39             if (idx != INVALID_IDX && md_.flag_traverse_[idx] == md_.raycast_num_) break;
40             md_.flag_traverse_[idx] = md_.raycast_num_;
41         }
42     }
43     for (int i = 0; i < md_.cache_voxel_cnt_; ++i)
44     {
45         int buf_id = globalIdx2BufIdx(md_.cache_voxel_[i]);
46         double log_update = (md_.count_hit_[buf_id] > 0) ? mp_.prob_hit_log_ : mp_.
prob_miss_log_;
47         md_.count_hit_[buf_id] = md_.count_hit_and_miss_[buf_id] = 0;
48         md_.occupancy_buffer_[buf_id] =
49             std::min(std::max(md_.occupancy_buffer_[buf_id] + log_update, mp_.clamp_min_log_)
,
50                     mp_.clamp_max_log_);
51     }
52 }

```

这段实现的关键不是保存点云，而是通过 `flag_rayend_`、`flag_traverse_` 和 `cache_voxel_` 避免同一帧重复写体素。点云终点在局部地图内时按命中处理，超出边界时截断到窗口边界并作为空闲方向更新；最后统一写回 `occupancy_buffer_`，并用上下界截断控制 `log-odds` 数值范围。

4.4 障碍物膨胀

直接使用原始占据体素进行规划，会使轨迹过于贴近障碍物；同时飞行机器人本身具有三维尺寸，定位、控制和传感器测距也存在误差。因此，系统在占用地图上进行 **障碍物**

膨胀 (Dilation)，并维护单独的 `occupancy_buffer_inflate_` 供规划器查询。需要说明的是，源码中并没有分别存储“机体半径、定位误差、控制误差”等多个安全项，而是通过 `grid_map/obstacles_inflation` 这一总参数统一体现安全裕度。

膨胀距离到体素层数的转换在 `initMap()` 中完成：

$$n_{\text{inflate}} = \left\lceil \frac{d_{\text{inflate}} - 10^{-5}}{r_{\text{map}}} \right\rceil \quad (11)$$

其中 d_{inflate} 对应 `obstacles_inflation`, r_{map} 对应 `resolution`。实飞参数 `inflation_size=0.18m`、`map_resolution=0.09m`，约为 2 层体素膨胀。该值过小会降低安全裕度，过大则可能封闭窄通道。

源代码路径：`src/diff_planner/plan_env/src/grid_map.cpp`; `src/diff_planner/plan_env/include/plan_env/grid_map.h`

Listing 3: 膨胀地图更新摘录 (`grid_map.cpp` / `grid_map.h`)

```

1 void GridMap::clearAndInflateLocalMap()
2 {
3     for (int i = 0; i < md_.cache_voxel_cnt_; ++i)
4     {
5         Eigen::Vector3i idx = md_.cache_voxel_[i];
6         int buf_id = globalIdx2BufIdx(idx);
7         int inf_buf_id = globalIdx2InfBufIdx(idx);
8
9         if (md_.occupancy_buffer_inflate_[inf_buf_id] < GRID_MAP_OBS_FLAG
10             && md_.occupancy_buffer_[buf_id] >= mp_.min_occupancy_log_)
11             changeInfBuf(true, inf_buf_id, idx);
12         if (md_.occupancy_buffer_inflate_[inf_buf_id] >= GRID_MAP_OBS_FLAG
13             && md_.occupancy_buffer_[buf_id] < mp_.min_occupancy_log_)
14             changeInfBuf(false, inf_buf_id, idx);
15     }
16 }
17
18 inline void GridMap::changeInfBuf(const bool dir, const int inf_buf_idx,
19                                   const Eigen::Vector3i global_idx)
20 {
21     int inf_grid = mp_.inf_grid_;
22     md_.occupancy_buffer_inflate_[inf_buf_idx] += dir ? GRID_MAP_OBS_FLAG : -
        GRID_MAP_OBS_FLAG;
23     for (int x_inf = -inf_grid; x_inf <= inf_grid; ++x_inf)
24         for (int y_inf = -inf_grid; y_inf <= inf_grid; ++y_inf)
25             for (int z_inf = -inf_grid; z_inf <= inf_grid; ++z_inf)
26             {
27                 Eigen::Vector3i id_inf(global_idx + Eigen::Vector3i(x_inf, y_inf, z_inf));
28                 int id_inf_buf = globalIdx2InfBufIdx(id_inf);
29                 md_.occupancy_buffer_inflate_[id_inf_buf] += dir ? 1 : -1;
30             }
31 }

```

这说明膨胀不是精确 ESDF 距离计算，而是在占用体素周围维护三维离散邻域计数。规划模块只查询膨胀地图即可判断安全性；代价是安全裕度强依赖 `obstacles_inflation`、地图分辨率和点云更新及时性。

4.5 地图管理策略

源码采用局部环形缓冲区（ring buffer）维护占用地图，窗口随无人机位置移动，窗口外体素被清理并复用，从而避免全局地图随飞行时间增长。

- (1) **局部环形地图**：以机器人当前位置为中心维护局部占用地图。实飞配置采用 $5.5m \times 5.5m \times 2.0m$ 的局部更新半径，源码中实际地图窗口约为 $11m \times 11m \times 4m$ 。
- (2) **虚拟墙约束**：通过虚拟地面和虚拟天花板参数限制飞行高度。规划器查询地图时，低于地面边界或高于天花板边界的位置会被视为不可通行。
- (3) **动态衰减机制**：实飞配置中衰减时间为 $0.5s$ 。长时间未被新观测确认的体素会逐步降低占用概率，避免旧障碍物长期残留在局部地图中。

膨胀地图通过 `getInflateOccupancy()` 提供给 A* 和轨迹优化。接口中，地图外返回 0，虚拟地面和天花板之外返回 -1，因此局部地图范围、规划视野和高度边界需要配合设置。

Listing 4: 规划器查询膨胀地图接口（grid_map.h）

```

1 inline int GridMap::getInflateOccupancy(Eigen::Vector3d pos)
2 {
3     if (mp_.enable_virtual_wall_ &&
4         (pos(2) >= mp_.virtual_ceil_ || pos(2) <= mp_.virtual_ground_))
5         return -1;
6
7     if (!isInInfBuf(pos))
8         return 0;
9
10    return int(md_.occupancy_buffer_inflate_[
11        globalIdx2InfBufIdx(pos2GlobalIdx(pos))]);
12 }
```

表 1 建图模块关键工程参数

参数	取值	作用
地图分辨率	0.09m	体素地图的空间离散精度
局部更新范围	$5.5m \times 5.5m \times 2.0m$	以当前位置为中心的半边长参数
实际局部窗口	约 $11m \times 11m \times 4m$	环形缓冲区覆盖的局部地图范围
障碍物膨胀距离	0.18m	转换为约 2 层体素膨胀
动态衰减时间	0.5s	未持续观测到的占用体素逐步衰减
虚拟高度边界	0.3m–1.4m	限制飞行高度，作为地图层面的安全约束

这些策略把实时查询限制在局部环形地图内，并结合缓存、log-odds 截断、膨胀和动态衰减提供可更新的环境表示。需要注意的是，膨胀只作用于已观测并写入地图的障碍物，实飞中仍需配合合理速度和规划视野。

5 轨迹规划

5.1 概述

轨迹规划是飞行机器人实现自主导航的核心，其任务是在已知或未知环境中，为机器人生成一条从起点到目标点的安全、平滑、动力学可行的轨迹。本章介绍基于 **Diff-Planner** 算法的轨迹规划系统。该系统采用有限状态机管理飞行模式，结合全局图搜索和局部优化流程，能够在动态非结构化环境中生成可执行的飞行轨迹，并在后端进行动力学可行性检查。

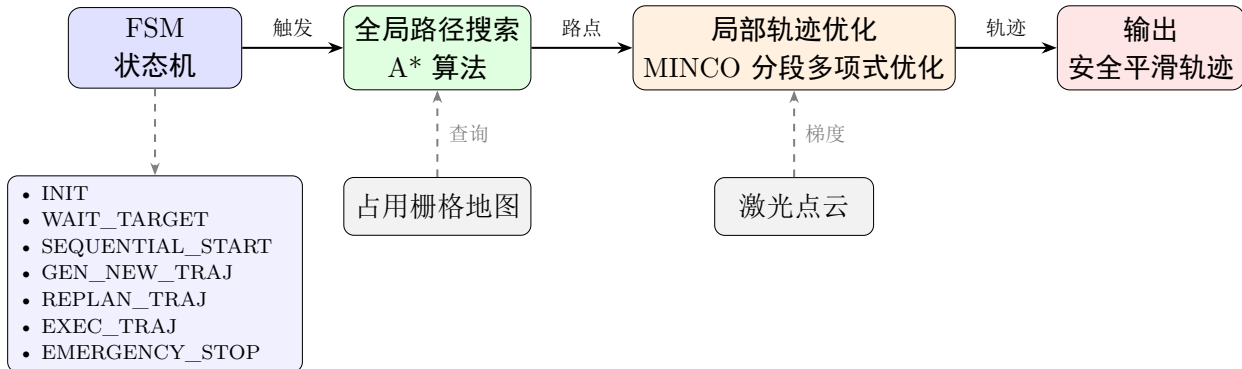


图 7 Diff-Planner 系统框架：FSM 状态管理 → 全局 A* 搜索 → 无预建 ESDF 的局部优化

5.2 系统框架

Diff-Planner 采用 **有限状态机（FSM）** 架构，将飞行过程划分为若干离散状态，每个状态下执行对应的规划策略。主要状态包括：

表 2 Diff-Planner 有限状态机

状态	描述
INIT	初始化，等待传感器数据与里程计就绪
WAIT_TARGET	待命，等待外部目标或触发指令
SEQUENTIAL_START	顺序启动，确保集群中前序无人机轨迹已收到后再开始规划
GEN_NEW_TRAJ	为新目标执行全局规划（A* + 优化）
REPLAN_TRAJ	局部重规划以避障，基于当前地图与状态
EXEC_TRAJ	执行当前轨迹并监测需要重规划的条件
EMERGENCY_STOP	紧急制动/悬停或触发故障保护

状态之间的转换由事件触发（如用户指令、到达目标、检测到障碍物等），由 FSM 统一管理，保证系统行为的一致性和可预测性。

Listing 5: Diff-Planner 有限状态机状态定义（diff_replan_fsm.h）

```

1 enum FSM_EXEC_STATE {
2     INIT,           // 初始化，等待传感器数据稳定
3     WAIT_TARGET,    // 待命，等待目标点指令
4     GEN_NEW_TRAJ,    // 接收新目标，执行全局规划
5     REPLAN_TRAJ,     // 检测到碰撞风险，执行局部重规划
6     EXEC_TRAJ,       // 执行当前轨迹
7     EMERGENCY_STOP,  // 紧急停止
8     SEQUENTIAL_START // 顺序启动（多点任务）
9 };
  
```

5.3 全局路径搜索

在已知地图（或已建图区域）中，采用 **A* 算法** 进行全局路径搜索，得到一条从起点到终点的粗略路径（路点序列）。A* 是图搜索中的经典算法，适合在离散栅格上快速生成几何可行的引导路径。

A* 算法的评价函数为：

$$f(n) = g(n) + h(n) \quad (12)$$

其中：

- $g(n)$: 从起点到节点 n 的实际累积代价（如路径长度、能耗）
- $h(n)$: 从节点 n 到终点的启发式估计代价。本系统采用对角线距离 (Diagonal Distance) 作为启发函数，并乘以一个很小的正系数 (tie-breaker)，以优先扩展更接近目标的节点。

为提高全局路径的安全性和可行性，本系统在标准 A* 的基础上引入了以下改进策略：

- (1) **26 邻域搜索**: 使用三维 26 连通邻域替代传统的 6 邻域，允许对角线移动，使搜索路径更短、更平滑。
- (2) **对角线启发式距离**: 采用对角线距离作为启发函数 $h(n)$ ，并乘以一个很小的正系数（代码中为 1.0001），以优先扩展更接近目标的节点，减少无谓的对称搜索。
- (3) **安全性约束**: 路径搜索中每个候选节点均通过膨胀占据栅格进行碰撞检测（`checkOccupancy()`），确保路径保持安全裕度。
- (4) **多轮隔离**: 通过 `rounds_` 计数器区分不同轮次搜索，避免跨轮数据污染。

全局路径规划采用改进的 A* 算法，支持动态障碍物更新。核心搜索循环如下：

Listing 6: 动态 A* 搜索算法 (dyn_a_star.cpp)

```
1 while (!openSet_.empty())
2 {
3     current = openSet_.top();
4     openSet_.pop();
5     if (current->index(0) == endPtr->index(0) &&
6         current->index(1) == endPtr->index(1) &&
7         current->index(2) == endPtr->index(2)) {
8         gridPath_ = retrievePath(current);
9         return ASTAR_RET::SUCCESS;
10    }
11    current->state = GridNode::CLOSEDSET;
12
13    for (int dx = -1; dx <= 1; dx++)
14        for (int dy = -1; dy <= 1; dy++)
15            for (int dz = -1; dz <= 1; dz++) {
16                if (dx == 0 && dy == 0 && dz == 0) continue;
17                Vector3i neighborIdx;
18                neighborIdx(0) = current->index(0) + dx;
19                neighborIdx(1) = current->index(1) + dy;
20                neighborIdx(2) = current->index(2) + dz;
21                if (neighborIdx(0) < 1 || neighborIdx(0) >= POOL_SIZE_(0) - 1 ||
22                    neighborIdx(1) < 1 || neighborIdx(1) >= POOL_SIZE_(1) - 1 ||
23                    neighborIdx(2) < 1 || neighborIdx(2) >= POOL_SIZE_(2) - 1)
24                    continue;
```

```

25     neighborPtr = GridNodeMap_[neighborIdx(0)][neighborIdx(1)][
neighborIdx(2)];
26     neighborPtr->index = neighborIdx;
27     bool flag_explored = neighborPtr->rounds == rounds_;
28     if (flag_explored && neighborPtr->state == GridNode::CLOSEDSET)
29         continue;
30     neighborPtr->rounds = rounds_;
31     if (checkOccupancy(Index2Coord(neighborPtr->index))) continue;
32
33     double static_cost = sqrt(dx*dx + dy*dy + dz*dz);
34     tentative_gScore = current->gScore + static_cost;
35     if (!flag_explored || tentative_gScore < neighborPtr->gScore) {
36         neighborPtr->cameFrom = current;
37         neighborPtr->gScore = tentative_gScore;
38         neighborPtr->fScore = tentative_gScore + getHeu(neighborPtr,
endPtr);
39
40         if (!flag_explored) {
41             neighborPtr->state = GridNode::OPENSET;
42             openSet_.push(neighborPtr);
43         }
44     }
45 }
46 return ASTAR_RET::SEARCH_ERR;
47 }

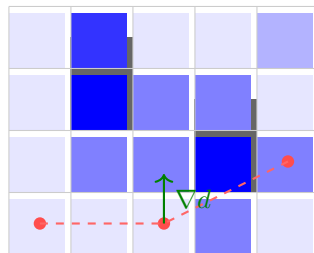
```

该实现的关键是：用 26 邻域提高三维搜索连通性，用对角线距离和 `tie_breaker_=1.0001` 减少对称扩展，用 `rounds_` 隔离不同轮次搜索，并在扩展邻居前通过 `checkOccupancy()` 早期剪枝。`step_size` 则在路径精度与搜索开销之间折中。

5.4 无 ESDF 轨迹优化

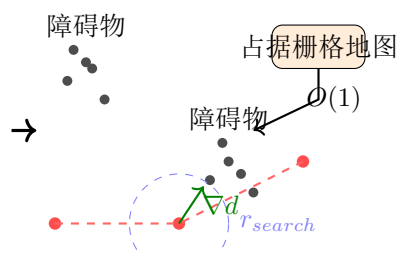
传统轨迹优化方法通常会引入 **ESDF** (Euclidean Signed Distance Field) 或其他距离场，用于估计轨迹点到障碍物的距离。距离场的构建和维护会带来额外计算与存储开销，尤其在大规模环境中更容易成为性能瓶颈。

传统方法：预建距离场



需预先构建全局距离场
带来额外存储与维护开销

本方法：无预建 ESDF



直接在占据栅格上进行碰撞检测
不依赖预建距离场

图 8 无预建 ESDF 优化原理对比：传统方法使用距离场（左），本方法直接在占据栅格上进行碰撞检测（右）

Diff-Planner 采用 **无预建 ESDF** 的优化策略，直接在占据栅格地图上进行碰撞检测，并结合 A* 辅助路径构造排斥方向，避免了距离场的预构建开销。具体流程如下：

(1) **碰撞检测**：对轨迹的每个采样控制点，通过膨胀占据栅格查询 (`getInflate`

Occupancy()) 判断是否发生碰撞，查询复杂度为 $O(1)$ 。

- (2) **约束点与排斥方向计算**: 对发生碰撞的轨迹段，通过 A* 搜索找到离散无碰路径，并据此构造排斥基准点 $\mathbf{p}_{base}$ 和排斥方向 $\mathbf{d}$ ，作为优化的约束条件。
- (3) **轨迹调整**: 在 L-BFGS 优化中，碰撞代价函数沿排斥方向”推动”控制点远离障碍物。目标函数由平滑项、碰撞项、目标项等组成。

$$\min_{\mathbf{p}} \lambda_s E_{smoothness} + \lambda_c E_{collision} + \lambda_d E_{goal} \quad (13)$$

其中 $E_{smoothness}$ 表示平滑项， $E_{collision}$ 表示碰撞项， E_{goal} 表示目标项。代码中还包含速度、加速度和加加速度等动力学罚项。

与传统方法相比的特点：

- **计算流程更直接**: 碰撞检测直接基于占据栅格完成，不需要预先维护全局距离场。
- **存储开销更低**: 不需要额外保存全局 ESDF 体素场，避免了距离场维护带来的资源占用。
- **适配动态环境**: 障碍物更新后可以直接基于占据栅格重新进行检测与优化。

MINCO 分段多项式参数化与优化流程 在实现层面，轨迹采用基于 MINCO 的分段多项式参数化。优化器以分段控制点和时间分配为主要变量，通过 MinJerkOpt 生成轨迹，并在 L-BFGS 中共同处理平滑、碰撞和动力学约束。

Listing 7: 碰撞代价与动力学检查摘录 (poly_traj_optimizer.cpp)

```

1 bool PolyTrajOptimizer::obstacleGradCostP(const int i_dp,
2     const Eigen::Vector3d &p, Eigen::Vector3d &gradp, double &costp) {
3     if (i_dp == 0 || i_dp > ConstraintPoints::two_thirds_id(cps_.points,
4         touch_goal_)) return false;
5     gradp.setZero(); costp = 0;
6     for (size_t j = 0; j < cps_.direction[i_dp].size(); ++j) {
7         double dist = (p - cps_.base_point[i_dp][j]).dot(cps_.direction[i_dp][j]);
8         double dist_err = obs_clearance_ - dist;
9         double dist_err_soft = obs_clearance_soft_ - dist;
10        Eigen::Vector3d dist_grad = cps_.direction[i_dp][j];
11        if (dist_err > 0) {
12            costp += wei_obs_ * pow(dist_err, 3);
13            gradp += -wei_obs_ * 3.0 * dist_err * dist_err * dist_grad;
14        }
15        if (dist_err_soft > 0) {
16            double r = 0.05;
17            double rsqr = r * r;
18            double term = sqrt(1.0 + dist_err_soft * dist_err_soft / rsqr);
19            costp += wei_obs_soft_ * rsqr * (term - 1.0);
20            gradp += -wei_obs_soft_ * dist_err_soft / term * dist_grad;
21        }
22    }
23    return costp > 0;
24 }
25
26 bool PolyTrajOptimizer::checkDynamicFeasibility(
27     const poly_traj::MinJerkOpt &pt_data) {
28     poly_traj::Trajectory traj = pt_data.getTraj();

```

```

29 Eigen::VectorXd durations = traj.get Durations();
30 const double RES = grid_map_->getResolution();
31 double t_step = min(RES / max_vel_,
32     durations.minCoeff() / max(cps_num_prePiece_, 1) / 1.5);
33 double traj_duration = traj.getTotalDuration();
34 for (double t = 0.0; t < traj_duration; t += t_step) {
35     if (traj.getVel(t).norm() > max_vel_ + vel_tolerance_)
36         return false;
37     if (traj.getAcc(t).norm() > max_acc_ + acc_tolerance_)
38         return false;
39 }
40 if (traj.getVel(traj_duration).norm() > max_vel_ + vel_tolerance_)
41     return false;
42 if (traj.getAcc(traj_duration).norm() > max_acc_ + acc_tolerance_)
43     return false;
44 return true;
45 }

```

以上摘录体现了两个实现要点：碰撞代价采用硬安全距离的三次惩罚与软安全距离的平滑惩罚并存；动力学可行性通过采样检查速度、加速度上界，作为轨迹发布前的最后防线。

5.5 轨迹弹开重规划机制

当 L-BFGS 检测到当前轨迹与障碍物冲突时，Diff-Planner 采用 **轨迹弹开重规划 (Rebound Replan)**：不是叠加速度矢量，而是重新构造约束点并通过梯度优化调整整条轨迹。

具体流程如下：

- (1) 用 `roughlyCheckConstraintPoints()` 查询膨胀占据栅格，定位碰撞轨迹段。
- (2) 在碰撞段两端执行 A* 搜索，得到离散无碰路径。
- (3) 由 A* 路径与当前控制点构造排斥基准点 $\mathbf{p}_{base}$ 和方向 $\mathbf{d}$ ，使碰撞代价沿该方向推动控制点远离障碍物。
- (4) 当 `allowRebound()` 满足迭代数 ≥ 3 、折角 $< 30^\circ$ 、初始障碍已规避等条件时，触发 `STOP_FOR_REBOUND` 并重启优化；若碰撞离当前时间过近，则直接紧急悬停。

5.6 安全保护机制

Diff-Planner 的安全保护集中在执行期监测与失败恢复：

- (1) 安全回调以 50ms (20Hz) 周期查询当前执行轨迹的膨胀占据栅格。
- (2) 若碰撞点距当前时间小于 `emergency_time_` (默认 1.0s)，立即生成悬停轨迹并切换至 `EMERGENCY_STOP`。
- (3) 若 `enable_fail_safe_` 开启且速度降至 0.1m/s 以下，则重新进入 `GEN_NEW_TRAJ`；若目标被占据，则沿全局轨迹回退寻找最近无碰目标。
- (4) 连续重规划失败超过 `MAX_REPLAN_FAIL_COUNT` (10 次) 或深度数据超时，均触发紧急悬停。

5.7 轨迹表示与优化

本系统采用 MINCO 分段多项式轨迹表示飞行轨迹。实现中以 `poly_traj::MinJerkOpt` 生成轨迹，并由分段内点与时间分配共同确定曲线形状。这里不再保留其他轨迹表示的数学表述，以免与当前代码实现混淆。

6 飞行控制

6.1 概述

飞行控制是飞行机器人实现稳定、精准飞行的执行层。如何将规划模块生成的理论轨迹转化为电机的实际转速，是飞行控制系统的核心任务。本章介绍基于 **PX4** 开源飞控的级联控制架构，包括位置控制、姿态控制和电机输出混合三个层次。该架构已在多旋翼飞行器上得到广泛验证，具有良好的稳定性和可调优性。

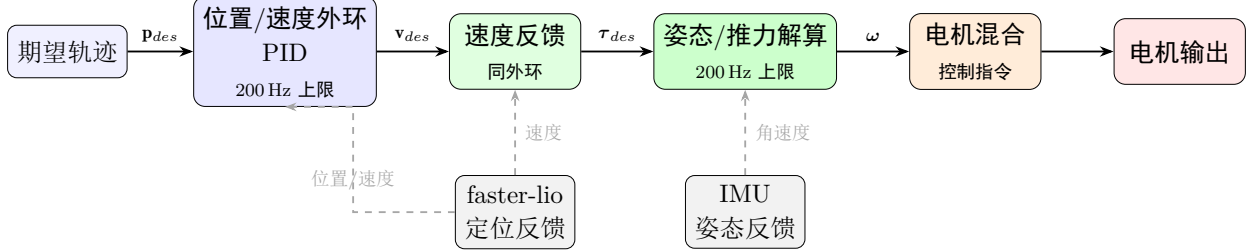


图 9 级联控制架构：位置外环、姿态控制器与传感器反馈回路

6.2 控制系统架构

本系统采用 **级联控制架构**，把轨迹跟踪拆成多个时间尺度更明确的子问题。信号流向为：规划轨迹输入 → 位置外环 → 姿态控制器 → 电机混合 → 电机输出；其中 faster-lio 提供的位置与速度反馈共同参与外环计算，但代码中并未单独实现一个独立的速度内环。

- (1) **位置外环**：把位置误差和速度反馈映射为期望加速度。
- (2) **速度反馈**：为位置外环提供速度修正量，并抑制位置误差带来的超调。
- (3) **姿态环**：根据姿态误差计算三轴期望扭矩。
- (4) **电机混合**：把总推力和三轴扭矩分配到四个电机。

源码中的主要运行周期：

- `traj_server` 指令发布：100Hz，由 `ros::Duration(0.01)` 定时触发。
- Diff-Planner FSM 执行：100Hz，由 `exec_timer` 定时触发。
- 安全碰撞检查：20Hz，由 `safety_timer` 定时触发。
- `px4ctrl` 主控制循环：上限 200Hz，由 `ctrl_freq_max=200.0` 指定。

较高频率的控制与安全检查能够更快抑制外部瞬时扰动，例如阵风或轻微碰撞，这也是系统稳定性的关键。

6.3 位置控制

位置控制器采用 **位置-速度外环策略**，将轨迹误差转化为期望加速度（在导航坐标系下）：

$$\mathbf{a}_{des} = K_p(\mathbf{p}_{des} - \mathbf{p}_{curr}) + K_v(\mathbf{v}_{des} - \mathbf{v}_{curr}) \quad (14)$$

其中：

- $\mathbf{p}_{des}, \mathbf{v}_{des}$ ：规划模块给出的期望位置和速度（来自 B 样条轨迹，通过求一阶导、二阶导得到期望速度和加速度）
- $\mathbf{p}_{curr}, \mathbf{v}_{curr}$ ：faster-lio 定位模块实时估计的当前位置和速度
- K_p, K_v ：位置与速度反馈增益，通过实地飞行调试确定

参数整定结果（源码控制模型中 `mass=1.0kg`，适用于当前 C5 飞行平台）：

- $K_p = 2.5$ （比例增益，决定响应速度；过大易震荡，过小响应迟缓）
- $K_v = 3.0$ （速度反馈增益，用于抑制超调并提升收敛速度）

值得注意的是，这里代码输出的不是完整的姿态控制量，而是一个经过限幅的**期望加速度**。也就是说，这一层做的是“位置误差 → 速度修正 → 加速度命令”的外环映射，实际实现中通过位置误差和速度误差的限幅抑制上层轨迹抖动，而不是单独再增加一个积分环节。

飞行控制器采用级联控制架构，核心逻辑很短：先处理当前位置目标是否有效，再对每个轴分别做位置误差和速度误差的限幅处理，最后输出加速度指令。

Listing 8: PID 误差加速度计算（controller.cpp）

```

1 Eigen::Vector3d Controller::computePIDErrorAcc(
2     const Odom_Data_t &odom,
3     const Desired_State_t &des,
4     const Parameter_t &param)
5 {
6     Eigen::Vector3d acc_error;
7
8     // X-axis PID control
9     double x_pos_error = std::isnan(des.p(0)) ? 0.0 :
10         std::max(std::min(des.p(0) - odom.p(0), 1.0), -1.0);
11     double x_vel_error = std::max(std::min(
12         (des.v(0) + Kp(0) * x_pos_error) - odom.v(0), 1.0), -1.0);
13     acc_error(0) = Kv(0) * x_vel_error;
14
15     // Y-axis PID control
16     double y_pos_error = std::isnan(des.p(1)) ? 0.0 :
17         std::max(std::min(des.p(1) - odom.p(1), 1.0), -1.0);
18     double y_vel_error = std::max(std::min(
19         (des.v(1) + Kp(1) * y_pos_error) - odom.v(1), 1.0), -1.0);
20     acc_error(1) = Kv(1) * y_vel_error;
21
22     // Z-axis PID control
23     double z_pos_error = std::isnan(des.p(2)) ? 0.0 :
24         std::max(std::min(des.p(2) - odom.p(2), 1.0), -1.0);
25     double z_vel_error = std::max(std::min(
26         (des.v(2) + Kp(2) * z_pos_error) - odom.v(2), 1.0), -1.0);
27     acc_error(2) = Kv(2) * z_vel_error;
28
29     return acc_error;
30 }

```

这段代码中：第一，`std::isnan(des.p(i))` 允许控制器在某些模式下不使用某一轴的位置目标，相当于把位置环的输入开关做成了运行时可配置。第二，位置误差和速度误差都被夹紧到 $[-1, 1]$ ，说明它并不追求“误差一次算满”，而是通过限幅避免上层轨迹抖动直接传递给飞控。

6.4 姿态控制

姿态控制器采用**四元数反馈控制**，直接处理姿态误差，避免了使用欧拉角表示时可能出现的万向节锁（Gimbal Lock）问题。

姿态误差通过四元数乘法计算：

$$\mathbf{q}_{err} = \mathbf{q}_{des} \otimes \mathbf{q}_{curr}^{-1} \quad (15)$$

将姿态误差 $\mathbf{q}_{err} = [q_w, q_x, q_y, q_z]^T$ 转换为轴角表示后，误差向量为 $\mathbf{e}_q = [q_x, q_y, q_z]^T \cdot 2 \arcsin(q_w) / \|[q_x, q_y, q_z]\|$ 。姿态控制扭矩计算公式为：

$$\boldsymbol{\tau}_{des} = K_p \mathbf{e}_q + K_d \boldsymbol{\omega}_{err} \quad (16)$$

其中：

- $\mathbf{e}_q \in \mathbb{R}^3$ ：姿态误差向量（由四元数误差转换而来）
- $\boldsymbol{\omega}_{err} = \boldsymbol{\omega}_{des} - \boldsymbol{\omega}_{curr} \in \mathbb{R}^3$ ：机体角速度误差
- K_p, K_d ：姿态控制的 PD 增益；当前源码配置未设置独立姿态环 PD 增益，速率控制相关配置为 KAngR=20.0、KAngP=20.0、KAngY=4.0

设计要点：姿态控制的采样频率必须足够高，否则在高速机动时可能出现姿态失稳。当前源码中 px4ctrl 主控制循环上限为 200Hz，姿态/推力解算随该循环执行。

6.5 偏航角控制策略

偏航角（Yaw，即机头朝向）控制与滚转（Roll）、俯仰（Pitch）控制有所不同。在四旋翼飞行器中，偏航运动不影响飞行器的位移（假设螺旋桨对称），只影响机头朝向和侧向视野。本系统根据不同的飞行模式采用差异化的偏航控制策略：

- (1) **定点模式：**偏航角锁定为用户设定的固定方向（如机头始终朝前），便于遥操作控制，也避免动力学建模时的侧向速度耦合。
- (2) **探索模式：**偏航角跟随飞行方向（即轨迹的切线方向），最大化激光雷达和相机的前方视野覆盖率，提高建图质量。
- (3) **精准控制模式：**偏航角变化率限制为 $90^\circ/\text{s}$ ，避免剧烈旋转导致 faster-lio 定位算法因运动模糊而失准，或视觉传感器图像过度运动模糊。

偏航角控制的模式切换由 Diff-Planner 的有限状态机触发，无需人工干预，实现了”模式感知的偏航控制”。

6.6 电机输出混合

四旋翼飞行器的电机输出混合（Mixer）把期望总推力 T_{des} 和三轴扭矩 $\tau_{roll}, \tau_{pitch}, \tau_{yaw}$ 分配到四个电机。对于“X”型四旋翼（C5 平台采用此布局），混合关系如下：

$$\begin{bmatrix} \omega_1^2 \\ \omega_2^2 \\ \omega_3^2 \\ \omega_4^2 \end{bmatrix} = \begin{bmatrix} 1 & 1 & -1 & 1 \\ 1 & -1 & 1 & 1 \\ 1 & 1 & 1 & -1 \\ 1 & -1 & -1 & -1 \end{bmatrix} \begin{bmatrix} T_{des} \\ \tau_{roll} \\ \tau_{pitch} \\ \tau_{yaw} \end{bmatrix} \quad (17)$$

其中 ω_i 是第 i 个电机的转速，平方与推力近似成正比。这个线性关系主要在低速范围内成立，高速时需要做非线性补偿。电机转速与推力的关系通过静态 thrust 测试标定得到。

电源管理策略：源码中低电压阈值为 low_voltage=21.0V。在推力映射时，若输入电压低于该阈值，控制器将用于计算的电压钳制到 21.0V 并输出低电压告警，以避免推力估计因电压异常继续降低；悬停油门基线采用 hover_percentage=0.45。

6.7 实际飞行表现

在室内和室外试验场中，我们以 faster-lio 的定位结果作为参考轨迹，采用更易复现的分场景测试方法对控制系统进行验证：

- **悬停精度测试：**在室内无风环境下进行 30s 悬停实验，统计位置和姿态误差以评估稳态控制精度。由飞行日志与 faster-lio 参考轨迹离线对齐后的统计结果表明，悬停实验中位置误差 RMS 小于 0.12m、姿态误差 RMS 小于 2°。
- **轨迹跟踪测试：**在 0.8m/s 的低速航段上执行预设轨迹跟踪，计算整段轨迹的跟踪误差。统计结果表明，在该速度条件下轨迹跟踪误差 RMS 小于 0.25m。
- **抗扰动测试：**在机体侧后方使用手持风扇施加短时扰动，记录最大位置偏移以及恢复到稳定悬停所需的时间。统计结果表明，在短时风扰条件下位置偏移峰值小于 0.4m，且可在 3s 内恢复稳定悬停。
- **动态响应测试：**在起飞、加速和转弯等机动过程中观察姿态变化与稳定时间。统计结果表明，在常规加速过程中姿态超调量小于 8°、稳定时间小于 2s。

以上数据表明，级联 PID 控制架构在合理整定参数后，能够满足比赛对精度和动态性能的要求。未来的改进方向包括引入 **模型预测控制 (MPC)** 以更好地处理动力学约束（如电机饱和、姿态角限制），以及利用 **在线系统辨识** 实现控制参数的自适应调整（适应电池电压下降、载重变化等场景）。

7 系统级调参与实飞适配分析

7.1 调参背景与总体原则

赛事平台已包含飞控、机载计算、雷达感知和规划控制框架。原始代码能够运行，但起飞高度、起降速度、状态切换间隔、局部地图边界和规划安全距离仍需结合比赛节奏复核。本章只说明两类参数：实际修改的项目，以及验证后选择沿用的项目。

参数适配遵循三个原则：先保证控制、定位、建图和规划分层稳定；只围绕明确目标修改参数；对已经与平台匹配的参数保留安全边界，避免为了节省少量时间而引入误触发或接管风险。

实际修改主要集中在 px4ctrl 的自动起降和状态机逻辑中。原因很直接：起飞、悬停、任务接管和降落会影响整条任务流程；若这些基础节奏不合适，即使建图和规划正常，也会浪费时间或影响接管稳定性。

7.2 控制与起降参数的适配修改

实际修改记录主要涉及以下文件：

- src/realflight_modules/px4ctrl/config/ctrl_param_fpv.yaml
- src/realflight_modules/px4ctrl/src/PX4CtrlFSM.h
- src/realflight_modules/px4ctrl/src/PX4CtrlFSM.cpp

这些文件位于控制链路末端，直接影响位置指令如何被 PX4 执行。原始参数能支持基本自动起降，但竞速任务要求更快进入任务段并在结束后快速退出，因此优先适配这部分。

7.2.1 自动起飞高度与起降速度

ctrl_param_fpv.yaml 中与自动起降相关的关键参数如下：

Listing 9: 自动起降参数配置 (ctrl_param_fpv.yaml)

```
1 auto_takeoff_land:
```

```

2 enable: true
3 enable_auto_arm: true
4 no_RC: false
5 takeoff_height: 0.6 # m
6 takeoff_land_speed: 0.2 # m/s

```

其中，takeoff_height=0.6m 决定起飞完成高度，takeoff_land_speed=0.2m/s 决定自动起降竖直速度。前者在节省爬升时间和避开贴地扰动之间折中；后者用于缩短起降等待，同时避免落地冲击过大。

表 3 自动起降参数修改原因

参数	调整后系数	修改目的	保留的边界
起飞高度	0.6m	减少起飞爬升时间，更快进入任务高度	高于贴地扰动区域，便于规划接管
起降速度	0.2m/s	缩短起飞和降落过程，提高比赛流程效率	速度不过激，避免明显冲击和姿态波动

7.2.2 电机预转时间与起飞后延时

PX4CtrlFSM.h 中定义了自动起降状态机的两个时间常量：

Listing 10: 自动起降状态机时间常量（PX4CtrlFSM.h）

```

1 static constexpr double MOTORS_SPEEDUP_TIME = 3.0;
2 static constexpr double DELAY_TRIGGER_TIME = 2.0;

```

电机预转时间为 3.0s，起飞后延时为 2.0s。这两个时间没有继续压缩，因为它们分别用于等待电机/飞控状态稳定和起飞后姿态收敛，直接关系到规划器接管时的初始状态。

7.2.3 状态切换重试间隔

PX4CtrlFSM.cpp 通过 last_trial_time 限制 PX4 状态切换、解锁或上锁服务的重复调用。当前源码中相关服务调用间隔为 1.0s，用于避免状态切换请求过密。

表 4 状态切换重试间隔调整分析

调参对象	原始设计目的	实际调整原因	注意事项
状态切换重试间隔	避免过于频繁调用 PX4 服务	当前源码阈值为 1.0s	保留间隔，防止重复请求过密

这些修改的共同目标是减少起降和状态切换中的无效等待，同时保留起飞稳定、服务调用频率和落地冲击等安全边界。

7.3 经验证后选择沿用的关键参数

除实际修改项外，我们也检查了建图、规划、定位和控制中的关键参数。部分原始参数已与当前平台、速度范围和低空场景匹配；在没有异常现象或重新标定依据时，沿用比盲目改动更稳妥。

7.3.1 建图与膨胀参数的沿用

当前 LIO 实飞链路中，建图相关关键参数如下：

表 5 经验证后沿用的建图参数

参数	当前值	沿用原因	继续关注的问题
map_resolution	0.09m	可提供较细的局部体素表达，适合室内障碍物避让	分辨率过小会增加计算量
inflation_size	0.18m	对应约 2 层体素膨胀，能够提供基本安全裕度	换机体尺寸或速度后需重新评估
fading_time	0.5s	可减少旧障碍物在局部地图中长期残留	点云短时丢失时可能清除真实障碍
max_ray_length	4.5m	与局部地图和室内雷达有效感知相匹配	高速飞行时可能需要更远感知
virtual_ground	0.3m	限制贴地飞行，减少地面附近风险	设置过高会压缩飞行空间
virtual_ceil	1.4m	符合低空室内飞行的高度约束	场地限高变化时要调整

其中 $\text{inflation_size}=0.18\text{m}$ 与 $\text{map_resolution}=0.09\text{m}$ 对应约 2 层体素膨胀：

$$n_{\text{inflate}} = \left\lceil \frac{0.18 - 10^{-5}}{0.09} \right\rceil = 2$$

该距离可为机体尺寸、定位误差和控制误差预留基本余量，同时不会过度压缩通道，因此作为当前速度范围和局部地图分辨率下的基线参数保留。

7.3.2 规划速度与安全距离参数的沿用

规划层重点检查速度、加速度、规划视野和障碍物距离代价，目的是确认轨迹不会因过保守而堵住窄通道，也不会因过激进而难以跟踪。

表 6 经验证后沿用的规划与优化参数

参数	当前值	沿用原因	继续关注的问题
max_vel	0.8m/s	对当前感知更新和控制能力较保守，适合初期实飞	后续提速必须同步验证控制误差
max_acc	3.0m/s ²	保证轨迹不过于激进，便于 PX4Ctrl 跟踪	过低会降低机动性
max_jer	25.0m/s ³	保持轨迹平滑和响应速度之间的折中	高速场景需重新验证
planning_horizon	7.5m	与局部地图范围和雷达感知距离基本匹配	地图射线有效长度仍需注意
硬安全距离	0.16m	与地图膨胀共同提供硬安全约束	不能单独看，需结合膨胀距离
软安全距离	0.5m	让优化轨迹主动远离障碍物	过大可能导致窄通道规划困难

地图层膨胀和优化层安全距离不是同一参数：前者扩大障碍体素，后者作为连续轨迹优化中的距离代价。两者叠加提供安全裕度，但任一项过大都会压缩可通行空间或增加优化难度，因此当前组合先作为稳定基线保留。

7.3.3 Faster-LIO 与外参参数的沿用

Faster-LIO 的雷达话题、IMU 话题、时间戳尺度和外参直接影响定位稳定性。当前配置与 MID-360 驱动输出和平台安装关系一致，因此在没有重新标定依据时选择沿用。

表 7 经验证后沿用的定位相关参数

参数	当前值	沿用原因	继续关注的问题
lid_topic	/livox/lidar	与 Livox 驱动输出一致	换雷达驱动需修改
imu_topic	/livox/imu	使用雷达 IMU，时间关系较清晰	若改用飞控 IMU 需重配
time_scale	10^{-9}	MID-360 使用纳秒时间戳	换驱动后必须确认
extrinsic_T	$[-0.011, -0.02329, 0.04412]$	与当前雷达安装位置对应	传感器重装后必须重测
extrinsic_R	$[1, 0, 0; 0, 1, 0; 0, 0, 1]$	当前安装方向下可用	雷达倾角变化必须重测
point_filter_num	3	在点云密度和计算量之间折中	细障碍漏检时可减小
iVox 分辨率	0.5	控制 LIO 地图搜索复杂度	定位精度不足时可评估

其中外参尤其不宜凭经验修改，否则可能让点云与机体姿态错位，进而影响建图和规划。

7.3.4 控制模型参数的沿用

控制参数直接影响轨迹跟踪。当前配置可提供稳定低速飞行基线，检查重点是确认质量、悬停油门和增益不会与规划速度约束冲突。

表 8 经验证后沿用的控制模型参数

参数	当前值	沿用原因	继续关注的问题
mass	1.0kg	与源码控制模型一致，可作为控制基准	换电池或加装载荷后需复核
hover_percentage	0.45	悬停油门处于合理范围	悬停油门明显偏离时需重标定
max_angle	80deg	保留较大姿态能力上限	实飞中仍由轨迹速度和控制器约束间接限制
Kp0--Kp2	2.5	位置环响应较稳健	震荡或迟滞时再调
Kv0--Kv2	3.0	速度环提供基本阻尼	高速跟踪误差大时检查

本阶段目标是稳定低速飞行和任务闭环，而不是控制性能极限调参。若后续提速或增加载荷，再对质量、悬停油门和控制增益进行系统辨识更合理。

7.4 参数之间的联动关系

实飞调参不能孤立看单个数值。下面将起飞高度、膨胀距离和速度上限换算为可比较的裕量，用于说明修改与沿用的依据。

7.4.1 起飞高度与虚拟墙

自动起飞高度需要落在虚拟地面与虚拟天花板之间。当前起飞高度 $0.6m$ 距虚拟地面 $0.3m$ 、距虚拟天花板 $0.8m$ ，既节省爬升时间，也保留上下方向规划余量。

表 9 起飞高度与虚拟墙裕量复核

项目	数值	计算关系	结论
虚拟地面	$0.3m$	建图模块下边界约束	避免贴地规划
起飞高度	$0.6m$	自动起飞完成高度	低高度、快进入任务
虚拟天花板	$1.4m$	建图模块上边界约束	限制过高飞行
下方裕量	$0.3m$	$0.6 - 0.3 = 0.3m$	不贴近虚拟地面
上方裕量	$0.8m$	$1.4 - 0.6 = 0.8m$	后续轨迹仍有空间

7.4.2 膨胀距离与可通行空间

障碍物膨胀不是越大越安全。当前 $0.18m$ 对应 2 层体素，能兼顾避障安全和通道通过能力：

$$n_{\text{inflate}} = \left\lceil \frac{0.18 - 10^{-5}}{0.09} \right\rceil = 2$$

若通道两侧都有障碍物，仅地图膨胀就会减少约 $2 \times 0.18 = 0.36m$ 可通行宽度；再叠加规划层硬安全距离 $0.16m$ ，余量会进一步缩小。该估算用于判断通道是否被安全参数压得过窄。

表 10 膨胀距离对可通行宽度的影响估算

原通道宽度	仅地图膨胀后	叠加硬安全距离后	判断
$0.8m$	约 $0.44m$	约 $0.12m$	已经偏紧，不宜继续增大膨胀
$1.0m$	约 $0.64m$	约 $0.32m$	仍可通过，但需要控制跟踪稳定
$1.2m$	约 $0.84m$	约 $0.52m$	通行余量较好

因此当前 $0.18m$ 膨胀距离保留基本安全裕度，也没有把通道过度压缩。

7.4.3 速度上限与感知更新

提高 `max_vel` 会缩短感知、建图、重规划和控制响应的的时间裕量。当前规划视野为 $7.5m$ ，地图射线有效长度为 $4.5m$ ；速度越高，可用于发现障碍物和修正轨迹的时间越少。

表 11 不同速度下的感知与规划时间裕量

速度上限	7.5m 规划视野	4.5m 射线长度	0.5s 内飞行距离
$0.8m/s$	约 $9.4s$	约 $5.6s$	$0.40m$
$1.2m/s$	约 $6.3s$	约 $3.8s$	$0.60m$
$1.5m/s$	约 $5.0s$	约 $3.0s$	$0.75m$

因此本阶段沿用 $0.8m/s$ 作为稳定基线，优先通过起降流程和状态切换间隔节省比赛时间。

7.5 本章小结

本章的实际修改集中在 `px4ctrl` 自动起降和状态机逻辑中：起飞高度 $0.6m$ 、起降速度 $0.2m/s$ 、电机预转时间 $3.0s$ 、起飞后延时 $2.0s$ ，以及状态切换重试间隔 $1.0s$ 。建图、规划、定位和控制中的关键参数则在理解源码作用和平台约束后选择沿用。后续若提高速度、改变载荷或调整传感器安装，需要重新用实飞日志和任务成功率验证这些参数边界。

8 总结

8.1 系统方案总结

本报告面向空中具身智能赛的自主飞行任务。由于比赛提供了成品飞行器及其软硬件平台，我们的工作主要集中在理解、适配与集成这些平台组件，并在此基础上实现和验证感知—建图—规划—控制的闭环：使用赛事平台上的 C5 飞行器及其机载计算单元、激光雷达、视觉传感器、双 IMU 与 PX4 飞控，我们实现了定位模块的在线估计、局部建图、轨迹规划与飞控接口的对接。

从工程实现的角度出发，我们更关注在给定平台上实现模块间的实时协同与可部署性，而非提出新的底层硬件设计。在实现中，Faster-LIO 提供高频状态估计；占用栅格与 Raycast 的地图更新为避障与搜索提供统一表达；Diff-Planner 将全局 A* 与无 ESDF 的局部优化结合，在保证安全裕度的同时控制计算开销；级联 PID 完成轨迹跟踪与姿态稳定。上述设计便于在赛事提供的机载算力与功耗约束下进行部署与调试。

8.2 技术特点与工程价值

我们认为本系统具有如下工程价值：

- (1) **模块链路覆盖完整**：我们实现了从硬件、状态估计、建图、规划到控制的端到端链路，能够将感知输入最终转化为执行指令，便于系统集成与现场调试。
- (2) **以实时性为导向的算法选型**：在各模块的实现中优先选用了计算开销可控且易部署的方案，以满足机载平台的实时运行要求。
- (3) **从感知到控制的安全约束**：我们在多个层面引入了保护机制——传感器冗余、地图膨胀、轨迹碰撞检测与紧急制动、控制限幅——以降低实飞风险。
- (4) **面向竞赛场景的可复现性**：硬件平台由比赛方提供，我们在该平台基础上对传感器配置与软件参数进行了适配，模块接口与运行频率的设计考虑了赛场条件，便于在赛场环境下快速复现与优化。

与仅呈现某一算法性能的工作不同，我们更关注系统级的可实现性与可复现性，这为后续在真实平台上开展对比实验和性能优化提供了基础。

8.3 现阶段不足

在实现与测试过程中，我们也发现了若干需要改进的地方：

- 实验记录不够规范：当前给出了一些性能指标（如定位精度、规划延迟、建图频率、控制误差），但测试条件、数据采集流程与对照基线尚需补充，影响结论的可重复性。
- 动态场景适应性有限：对于快速移动的障碍物或高度动态的环境，当前的检测与预测能力还有提升空间，可能导致重规划触发延迟或轨迹切换不平滑。
- 参数依赖经验调优：部分关键参数仍通过人工经验设定，缺乏系统化的辨识或自动调参流程，影响在不同平台或任务下的迁移性。
- 感知噪声影响优化精度：点云稠密度与噪声会影响无 ESDF 梯度估计的准确性，进而影响局部优化效果。

8.4 后续优化方向

接下来我们计划沿着以下方向推进：

- (1) 建立标准化测试流程，完整记录测试条件、传感器配置与失败案例，以提升性能结论的可重复性与可信度；
- (2) 引入对动态障碍物的运动估计与预测，使规划在时域上更具前瞻性，减少重规划频率与轨迹切换带来的抖动；
- (3) 在控制端探索基于模型的前馈或 MPC 元件，以改善高速机动下的跟踪精度与鲁棒性；
- (4) 完善工程文档与参数表，建立自动化调参或简单的系统辨识流程，降低现场调试成本。

我们认为，目前已在比赛提供的平台上完成了系统集成与初步验证，下一步应将工作重点放在以证据支撑设计决策上：通过更严谨的实验设计与场景覆盖，验证系统在复杂赛道中的稳健性与实时边界，并在必要时调整软件参数以适配不同赛场条件。